AI-Powered Fake News Detection Using Text Classification

Arnav Chawla
2nd Year, Bennett University

1. Introduction

The rapid proliferation of digital media has led to the widespread dissemination of misinformation, commonly referred to as "fake news." This phenomenon poses a significant threat to public discourse, political stability, and societal trust. The sheer volume of articles published daily makes manual fact-checking an impossible task. Therefore, there is a critical need for automated systems capable of identifying and filtering deceptive content. This project addresses this problem by developing an intelligent pipeline that utilizes Machine Learning (ML) and Natural Language Processing (NLP) to classify news articles as either real or fake with high accuracy.

2. Dataset Description

- **Source:** The data consists of two datasets originally curated for fake news detection research, representing real and fake news articles.
- **Size:** The combined dataset contains 44,898 articles (approximately 21,417 real and 23,481 fake).
- **Features:** The primary feature utilized for classification is the textual content of the articles (the `text` column). A target variable `label` was engineered, where `0` represents Fake News and `1` represents Real News.

3. Methodology

The pipeline consists of three main stages: Preprocessing, Feature Extraction, and Modeling.

A. Preprocessing

Raw text is highly unstructured. To prepare the data for the algorithms, a rigorous cleaning function was applied to every article:

- Lowercasing: Converting all text to lowercase to normalize vocabulary.
- URL Removal: Using regular expressions to strip out hyperlinks.
- Punctuation & Digit Removal: Eliminating numbers and punctuation marks to isolate semantic meaning.

B. Feature Extraction

The cleaned text was vectorized using TF-IDF (Term Frequency-Inverse Document Frequency). This mathematical statistic evaluates how important a word is to a document

within the entire dataset. The vocabulary was restricted to the top 5,000 most significant features to optimize computational performance.

C. Algorithms

The dataset was split into an 80% training set and a 20% testing set. Four algorithms were deployed:

1. 1. K-Nearest Neighbors (KNN)
2. 2. Logistic Regression
3. 3. Random Forest Classifier
4. 4. Neural Network (MLP Classifier)

4. Results

The models were evaluated on the 20% testing set (8,980 samples). The primary metrics used were Accuracy, Precision, Recall, F1-Score, and Confusion Matrices.

Performance Summary Table

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	99.74%	99.74%	99.72%	99.73%
Neural Network	98.67%	98.65%	98.58%	98.61%
Logistic Regression	98.59%	98.12%	98.93%	98.52%
K-Nearest Neighbors	70.50%	92.56%	41.50%	57.31%

Confusion Matrices

(Rows = Actual Label, Columns = Predicted Label)

KNN:
[4553, 143]
[2506, 1778]

Logistic Regression:
[4615, 81]
[46, 4238]

Random Forest:
[4685, 11]
[12, 4272]

Neural Network:
[4638, 58]
[61, 4223]

5. Discussion

In machine learning, models are broadly categorized as parametric or non-parametric. This project provides a clear comparison between the two approaches on high-dimensional text data.

Parametric Models (Logistic Regression, Neural Network):

These models make assumptions about the underlying distribution of the data and use a fixed number of parameters. Both Logistic Regression and the Neural Network performed exceptionally well (over 98% accuracy). This indicates that the TF-IDF feature space is highly linearly separable, allowing weight-based parametric models to easily distinguish between fake and real vocabulary.

Non-Parametric Models (KNN, Random Forest):

These models do not make strong assumptions about the data distribution and adapt their complexity to the training data.

- Random Forest was the best-performing model overall (99.74%). As an ensemble of decision trees, it is highly adept at handling sparse, non-linear, high-dimensional TF-IDF matrices without overfitting.
- KNN, conversely, was the worst-performing model (70.50%). Because it relies on distance metrics (like Euclidean distance), it suffers heavily from the "curse of dimensionality" when processing 5,000 features, leading to extremely poor recall (41.50%) for the positive class.

6. Conclusion

The project successfully demonstrates that machine learning and NLP can be powerfully leveraged to detect fake news. Ensemble non-parametric models (Random Forest) proved to be the most efficient and accurate for this specific text classification task, though simple parametric models (Logistic Regression) offer an excellent balance of high accuracy and fast training times.

Limitations: The model relies on specific vocabulary patterns from a static dataset, meaning it may struggle to classify newly emerging slang or entirely novel events without retraining.

Future Scope: Future iterations could involve integrating real-time web scraping to evaluate live news articles, or utilizing deep learning transformer models (like BERT) to understand context rather than just word frequencies.

7. Appendix

Python Code

The full pipeline code is available in the Fake_News_Detection.ipynb file in the project directory, implementing TF-IDF vectorization and 4 different Scikit-Learn models.

Test Data Samples

Fake News Sample (Label = 0):

Title: "Trump drops Steve Bannon from National Security Council"
Text: "21st Century Wire says Ben Stein, reputable conservative..."

Real News Sample (Label = 1):

Title: "WASHINGTON (Reuters) - U.S. President Donald Trump..."
Text: "WASHINGTON (Reuters) - U.S. President Donald Trump heads for Scotland..."